

LAPORAN PRAKTIKUM PEKAN 7

STRUKTUR DATA

“InsertionSort, SelectionSort, BubbleSort, dan InsertionGUP”

disusun Oleh:

NABIL FIKRI

2511533011

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

SHERLY SUKMADIRA



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum mata kuliah **Struktur Data** ini tepat pada waktunya.

Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum mengenai implementasi kelas dan objek melalui studi kasus pembuatan *InsertionSort*, *SelectionSort*, *BubbleSort*, dan *InsertionGUI* menggunakan bahasa pemrograman Java. Penulis menyadari bahwa keberhasilan penyusunan laporan ini tidak terlepas dari bantuan berbagai pihak. Oleh karena itu, penulis ingin menyampaikan terima kasih kepada:

1. Dr. Wahyudi S.T,M.T, selaku dosen mata kuliah yang telah memberikan bimbingan teori di kelas.
2. Kakak asisten praktikum yang telah memberikan arahan dan bantuan teknis selama proses praktikum berlangsung di laboratorium.
3. Rekan-rekan mahasiswa angkatan 2025 Informatika Universitas Andalas yang telah saling mendukung dalam proses belajar.

Penulis menyadari bahwa laporan ini masih jauh dari kesempurnaan, baik dari segi materi maupun sistematika penulisannya. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan perkembangan ilmu pengetahuan, khususnya di bidang Informatika.

Padang, 18 Mei 2026

Nabil Fikri

DAFTAR ISI

KATA PENGANTAR	i
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	1
BAB II PEMBAHASAN	2
2.1 Mengenal macam-macam Sorting	2
2.1.1 Bubble Sort	2
2.2 Selection Sort.....	3
2.3 Insertion Sort	4
2.4 Implementasi pada Java	4
2.4.1 InsertionSort_2511533011.....	4
2.4.2 SelectionSort_2511533011	6
2.4.3 BubbleSort_2511533011	8
2.4.4 InsertionSort_2511533011.....	10
BAB III PENUTUP	20
3.1 Kesimpulan.....	20
3.2 Saran.....	20
DAFTAR PUSTAKA	21

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman dan struktur data, proses pengurutan data (sorting) merupakan salah satu teknik yang sangat penting. Pengurutan data digunakan untuk menyusun data agar lebih teratur sehingga memudahkan proses pencarian, pengolahan, dan analisis data. Banyak aplikasi komputer menggunakan teknik sorting, seperti pengurutan nilai, data mahasiswa, data penjualan, dan berbagai jenis data lainnya.

Terdapat berbagai algoritma sorting yang dapat digunakan, di antaranya Insertion Sort, Selection Sort, dan Bubble Sort. Masing-masing algoritma memiliki cara kerja, kelebihan, dan kekurangan yang berbeda. Insertion Sort bekerja dengan menyisipkan elemen ke posisi yang tepat, Selection Sort bekerja dengan memilih elemen terkecil pada setiap iterasi, sedangkan Bubble Sort bekerja dengan menukar elemen yang berdekatan hingga data terurut.

Pada praktikum ini dilakukan implementasi algoritma sorting menggunakan bahasa pemrograman Java. Selain membuat program sorting berbasis console, praktikum juga mengimplementasikan Insertion Sort berbasis GUI (Graphical User Interface) menggunakan Java Swing agar proses pengurutan dapat dilihat secara visual dan lebih mudah dipahami.

Melalui praktikum ini diharapkan mahasiswa dapat memahami konsep dasar sorting, cara kerja masing-masing algoritma, serta mampu mengimplementasikannya ke dalam program Java.

1.2 Tujuan Praktikum

1. Memahami konsep dasar algoritma pengurutan data (sorting).
2. Memahami cara kerja algoritma Insertion Sort, Selection Sort, Bubble Sort.
3. Mengimplementasikan algoritma sorting menggunakan bahasa pemrograman Java.
4. Membuat visualisasi proses Insertion Sort menggunakan GUI Java Swing.
5. Menganalisis hasil pengurutan data menggunakan beberapa metode sorting.

BAB II

PEMBAHASAN

2.1 Menenal macam-macam Sorting

2.1.1 Bubble Sort

Bubble Sort (metode gelembung) adalah metode/algorithm pengurutan dengan cara melakukan penukaran data dengan data sebelahnya secara terus menerus sampai bisa dipastikan dalam satu iterasi tertentu tidak lagi perubahan (data sudah terurut secara benar).

Berikut Langkah-langkah dari bubble sort :

1. Input data
2. Setiap data, misalnya data pertama akan dibandingkan dengan data yang ada di sebelahnya (dari data kedua sampai selesai).
3. Bila data pertama tersebut lebih besar dari data yang ada pada data sesudahnya, dilakukan penukaran tempat atau posisi data.
4. Demikian untuk data kedua sampai dengan data terakhir dilakukan dengan cara serupa.

Jika diilustrasikan seperti ini :

*Proses
Algoritma
Bubble Sort*

Angka : 3, 1, 4, 2, 8

Berikut Proses pengurutan keseluruhannya :

Proses pertama
1, 3, 4, 2, 8
1, 3, 4, 2, 8
1, 3, 2, 4, 8
1, 3, 2, 4, 8
Proses kedua
1, 3, 2, 4, 8
1, 2, 3, 4, 8
1, 2, 3, 4, 8
Proses ketiga
1, 2, 3, 4, 8
1, 2, 3, 4, 8

Daismabali.com

2.2 Selection Sort

Selection Sort adalah algoritma pengurutan data dengan cara membandingkan elemen yang sekarang dengan elemen yang berikutnya sampai dengan elemen terakhir. Jika ditemukan elemen lain yang lebih kecil dari elemen sekarang maka dicatat posisinya dan kemudian ditukar.

Algoritma selection sort akan memindai nilai terkecil dari suatu Kumpulan data dan jika ada, data tersebut akan diletakkan pada urutan pertama. Begitu selanjutnya untuk urutan kedua dan seterusnya.

Langkah-langkahnya seperti ini :

1. Melakukan pengecekan dimulai dari data pertama hingga ke-n.
2. Menentukan data dengan indeks minimum (jika ascending) atau maksimum (jika descending).
3. Menukarkan data dengan indeks minimum (jika ascending) atau maksimum (jika descending) dengan bilangan pertama ($i=1$) dari data tersebut.
4. Mengulangi Langkah di atas untuk sisa data bilangan berikutnya ($i=i+1$) sampai didapatkan urutan yang sesuai.

Ilustrasinya seperti berikut :

*Proses
Algoritma
Selection
Sort*

Angka Awal: 15, 16, 2, 9

Berikut Proses pengurutan keseluruhannya :

Proses pertama
2, 16, 15, 9

Proses kedua
2, 9, 15, 16

Proses ketiga
2, 9, 15, 16

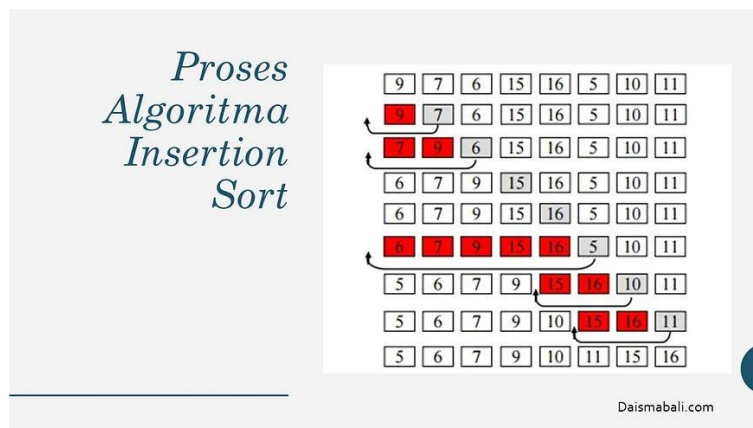
Daismabali.com

2.3 Insertion Sort

Insertion Sort adalah Sebuah algoritma pengurutan yang membandingkan dua elemen data pertama, mengurutkannya, kemudian mengecek elemen data berikutnya satu persatu dan membandingkannya dengan elemen data yang telah diurutkan.

Langkah-langkah :

1. Membandingkan dua elemen data pertama, mengurutkannya.
2. Mengecek elemen data berikutnya satu persatu.
3. Membandingkan dengan elemen data yang telah diurutkan.



2.4 Implementasi pada Java

2.4.1 InsertionSort_2511533011

```
package pekan7_2511533011;

public class SelectionSort_2511533011 {
    public static void SelectionSort_2511533011 (int[] arr_3011) {
        int n_3011 = arr_3011.length;
        for (int i_3011 = 0; i_3011 < n_3011; i_3011++) {
            int minIndex_3011 = i_3011;
            for (int j_3011 = i_3011 + 1; j_3011 < n_3011; j_3011++) {
                if (arr_3011[j_3011] < arr_3011[minIndex_3011]) {
                    minIndex_3011 = j_3011;
                }
            }
            // Swap logic would go here
        }
    }
}
```

```

    }
    }
    int temp_3011 = arr_3011[i_3011];
    arr_3011[i_3011] = arr_3011[minIndex_3011];
    arr_3011[minIndex_3011] = temp_3011;
}
}
public static void main(String[] args) {
    int arr_3011[] = {23,78, 45, 8, 32, 56, 1};
    int n_3011 = arr_3011.length;
    System.out.printf("array yang belum terurut :\n");
    for (int i_3011 = 0; i_3011 < n_3011; i_3011++)
        System.out.printf(arr_3011[i_3011] + " ");
    System.out.println("");
    SelectionSort_2511533011(arr_3011);
    System.out.printf("array yang terurut :\n");
    for (int i_3011 = 0; i_3011 < n_3011; i_3011++)
        System.out.print(arr_3011[i_3011] + " ");
    System.out.println("");
}
}

```

Kode Program 2.2.1

Class InsertionSort_2511533011 digunakan untuk mengurutkan data array menggunakan algoritma **Insertion Sort** secara ascending (kecil ke besar). Algoritma Insertion Sort bekerja dengan cara mengambil satu elemen, kemudian menyisipkannya ke posisi yang tepat pada bagian array yang sudah terurut.

- `public static void insertionSort(int[] arr_3011)`
Method ini digunakan untuk melakukan proses pengurutan array menggunakan algoritma Insertion Sort.
- `int n_3011 = arr_3011.length;`
Digunakan untuk menyimpan jumlah elemen array.
- `for (int i_3011 = 1; i_3011 < n_3011; i_3011++)`
Perulangan dimulai dari indeks ke-1 karena elemen pertama dianggap sudah terurut.
- `int key_3011 = arr_3011[i_3011];`

Variabel `key_3011` digunakan untuk menyimpan elemen yang akan disisipkan ke posisi yang benar.

- `int j_3011 = i_3011 - 1;`

Variabel `j_3011` digunakan untuk mengecek elemen sebelum `key_3011`.

- `while (j_3011 >= 0 && arr_3011[j_3011] > key_3011)`

Perulangan ini berjalan selama:

- indeks masih valid,
- dan nilai sebelumnya lebih besar dari `key_3011`.

Jika kondisi benar, maka elemen digeser ke kanan.

- `arr_3011[j_3011 + 1] = arr_3011[j_3011];`

memberikan ruang agar `key_3011` bisa dimasukkan pada posisi yang benar.

- `arr_3011[j_3011 + 1] = key_3011;`

Setelah proses pergeseran selesai, nilai `key_3011` dimasukkan ke posisi yang sesuai.

2.4.2 SelectionSort_2511533011

```
package pekan7_2511533011;

public class SelectionSort_2511533011 {
    public static void SelectionSort_2511533011 (int[] arr_3011) {
        int n_3011 = arr_3011.length;
        for (int i_3011 = 0; i_3011 < n_3011; i_3011++) {
            int minIndex_3011 = i_3011;
            for (int j_3011 = i_3011 + 1; j_3011 < n_3011;
j_3011++) {
                if (arr_3011[j_3011] <
arr_3011[minIndex_3011]) {
                    minIndex_3011 = j_3011;
                }
            }
            int temp_3011 = arr_3011[i_3011];
            arr_3011[i_3011] = arr_3011[minIndex_3011];
            arr_3011[minIndex_3011] = temp_3011;
        }
    }
}
```

```

public static void main(String[] args) {
    int arr_3011[] = {23,78, 45, 8, 32, 56, 1};
    int n_3011 = arr_3011.length;
    System.out.printf("array yang belum terurut :\n");
    for (int i_3011 = 0; i_3011 < n_3011; i_3011++)
        System.out.printf(arr_3011[i_3011] + " ");
    System.out.println("");
    SelectionSort_2511533011(arr_3011);
    System.out.printf("array yang terurut :\n");
    for (int i_3011 = 0; i_3011 < n_3011; i_3011++ )
        System.out.print(arr_3011[i_3011] + " ");
    System.out.println("");
}
}

```

Kode Program 2.2.2

Class SelectionSort_2511533011 merupakan program Java yang mengimplementasikan algoritma Selection Sort untuk mengurutkan data integer dalam array.

Algoritma Selection Sort bekerja dengan cara mencari elemen terkecil dari bagian array yang belum terurut, kemudian menukarnya dengan posisi awal.

- for (int i_3011 = 0; i_3011 < n_3011; i_3011++)

Perulangan utama digunakan untuk menentukan posisi elemen yang akan diisi dengan nilai terkecil.

Setiap iterasi:

- mencari nilai minimum,
- lalu menukarnya dengan elemen pada posisi saat ini.

- int minIndex_3011 = i_3011;

Variabel minIndex_3011 digunakan untuk menyimpan indeks nilai terkecil sementara. Awalnya dianggap bahwa elemen pada posisi i_3011 adalah nilai terkecil.

- for (int j_3011 = i_3011 + 1; j_3011 < n_3011; j_3011++)

Perulangan ini digunakan untuk memeriksa seluruh elemen setelah posisi i_3011.

- `if(arr_3011[j_3011] < arr_3011[minIndex_3011])`
Jika ditemukan nilai yang lebih kecil, maka indeks minimum diperbarui.
 - `minIndex_3011 = j_3011;`
program menyimpan posisi elemen terkecil terbaru.
 - `int temp_3011 = arr_3011[i_3011];`
`arr_3011[i_3011] = arr_3011[minIndex_3011];`
`arr_3011[minIndex_3011] = temp_3011;`
Bagian ini digunakan untuk menukar:
 - elemen pada posisi sekarang,
 - dengan elemen terkecil yang ditemukan.
- Variabel `temp_3011` digunakan sebagai tempat penyimpanan sementara agar data tidak hilang saat proses pertukaran.

2.4.3 BubbleSort_2511533011

```
package pekan7_2511533011;

public class BubbleSort_2511533011 {
    public static void BubbleSort_2511533011 (int[] arr_3011) {
        int n_3011 = arr_3011.length;
        for (int i_3011 = 0; i_3011 < n_3011; i_3011++) {
            for (int j_3011 = 0; j_3011 < n_3011 - i_3011 -
1; j_3011++) {
                if (arr_3011[j_3011] > arr_3011[j_3011 +
1]) {
                    int temp_3011 = arr_3011[j_3011];
                    arr_3011[j_3011] = arr_3011[j_3011
+ 1];
                    arr_3011[j_3011 + 1] = temp_3011;
                    // System.out.println("data : " + arr[j] +
" +arr[j+1]);
                }
            }
        }
    }
    public static void main(String[] args) {
        int arr_3011[] = {23,78, 45, 8, 32, 56, 1};
        int n_3011 = arr_3011.length;
```

```

        System.out.printf("array yang belum terurut :\n");
        for (int i_3011 = 0; i_3011 < n_3011; i_3011++)
            System.out.printf(arr_3011[i_3011] + " ");
        System.out.println("");
        BubbleSort_2511533011(arr_3011);
        System.out.printf("array yang terurut menggunakan
BubbleSort :\n");
        for (int i_3011 = 0; i_3011 < n_3011; i_3011++)
            System.out.print(arr_3011[i_3011] + " ");
        System.out.println("");
    }
}

```

Kode Program 2.2.3

Class BubbleSort_2511533011 merupakan program Java yang mengimplementasikan algoritma Bubble Sort untuk mengurutkan data integer dalam array.

Algoritma Bubble Sort bekerja dengan cara membandingkan dua elemen yang berdekatan, kemudian menukarnya jika urutannya salah. Proses ini dilakukan berulang-ulang sampai seluruh data terurut.

Disebut Bubble Sort karena nilai terbesar akan “mengapung” ke akhir array seperti gelembung.

- `int n_3011 = arr_3011.length;`
Digunakan untuk menyimpan jumlah elemen array.
- `for (int i_3011 = 0; i_3011 < n_3011; i_3011++)`
Perulangan luar digunakan untuk menentukan jumlah tahap pengurutan.
Pada setiap iterasi:
 - elemen terbesar akan berpindah ke posisi paling akhir.
- `for (int j_3011 = 0; j_3011 < n_3011 - i_3011 - 1; j_3011++)`
Perulangan dalam digunakan untuk membandingkan elemen yang berdekatan.
- `n_3011 - i_3011 - 1`

digunakan karena:

- bagian akhir array sudah terurut,
- sehingga tidak perlu dibandingkan lagi.
- `if (arr_3011[j_3011] > arr_3011[j_3011 + 1])`

Jika elemen kiri lebih besar daripada elemen kanan, maka kedua elemen ditukar.

- `int temp_3011 = arr_3011[j_3011];`
`arr_3011[j_3011] = arr_3011[j_3011 + 1];`
`arr_3011[j_3011 + 1] = temp_3011;`

Variabel `temp_3011` digunakan sebagai tempat penyimpanan sementara saat pertukaran data.

Tujuan:

agar nilai asli tidak hilang saat proses swap dilakukan.

2.4.4 InsertionSort_2511533011

```
package pekan7_2511533011;
```

```
import java.awt.BorderLayout;
```

```
import java.awt.Color;
```

```
import java.awt.Dimension;
```

```
import java.awt.FlowLayout;
```

```
import java.awt.Font;
```

```
import javax.swing.*;
```

```
import javax.swing.border.EmptyBorder;
```

```

public class InsertionSortGUI_2511533011 extends JFrame {

    private static final long serialVersionUID = 1L;

    private int[] array_3011;

    private JLabel[] labelArray_3011;

    private JButton stepButton_3011, resetButton_3011, setButton_3011;

    private JTextField inputField_3011;

    private JPanel panelArray_3011;

    private JTextArea stepArea_3011;

    private int i_3011 = 1, j_3011;

    private boolean sorting_3011 = false;

    private int stepCount_3011 = 1;

    /**
     * Create the frame.
     */

    public InsertionSortGUI_2511533011() {

        setTitle("Insertion Sort Langkah per Langkah");

        setSize(750, 400);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        setLocationRelativeTo(null);
    }

```

```
setLayout(new BorderLayout());

// Panel input

JPanel inputPanel_3011 = new JPanel(new FlowLayout());

inputField_3011 = new JTextField(30);

setButton_3011 = new JButton("Set Array");

inputPanel_3011.add(new JLabel("Masukkan angka (pisahkan dengan
koma):"));

inputPanel_3011.add(inputField_3011);

inputPanel_3011.add(setButton_3011);


// Panel array visual

panelArray_3011 = new JPanel();

panelArray_3011.setLayout(new FlowLayout());


// Panel kontrol

JPanel controlPanel_3011 = new JPanel();

stepButton_3011 = new JButton("Langkah Selanjutnya");

resetButton_3011 = new JButton("Reset");

stepButton_3011.setEnabled(false);

controlPanel_3011.add(stepButton_3011);
```

```
controlPanel_3011.add(resetButton_3011);

// Area teks untuk log langkah-langkah
stepArea_3011 = new JTextArea(8, 60);
stepArea_3011.setEditable(false);
stepArea_3011.setFont(new Font("Monospaced", Font.PLAIN, 14));
JScrollPane scrollPane_3011 = new JScrollPane(stepArea_3011);

// Tambahkan panel ke frame
add(inputPanel_3011, BorderLayout.NORTH);
add(panelArray_3011, BorderLayout.CENTER);
add(controlPanel_3011, BorderLayout.SOUTH);
add(scrollPane_3011, BorderLayout.EAST);

// Event Set Array
setButton_3011.addActionListener(e -> setArrayFromInput_3011());

// Event Langkah Selanjutnya\
stepButton_3011.addActionListener(e -> performStep_3011());

// Event reset
```



```

resetButton_3011.addActionListener(e -> reset_3011 ());

}

private void setArrayFromInput_3011() {

    String text = inputField_3011.getText().trim();

    if (text.isEmpty()) return;

    String[] parts = text.split(",");

    array_3011 = new int[parts.length];

    try {

        for (int k = 0; k < parts.length; k++) {

            array_3011[k] = Integer.parseInt(parts[k].trim());

        }

    } catch (NumberFormatException e) {

        JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang
dipisahkan "

            + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);

        return;

    }

    i_3011 = 1;

    stepCount_3011 = 1;

    sorting_3011 = true;

```

```

stepButton_3011.setEnabled(true);

stepArea_3011.setText("");

panelArray_3011.removeAll();

labelArray_3011 = new JLabel[array_3011.length];

for (int k = 0; k < array_3011.length; k++) {

    labelArray_3011[k] = new JLabel(String.valueOf(array_3011[k]));

    labelArray_3011[k].setFont(new Font("Arial", Font.BOLD, 24));

    labelArray_3011[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));

    labelArray_3011[k].setPreferredSize(new Dimension(50, 50));

    labelArray_3011[k].setHorizontalAlignment(SwingConstants.CENTER);

    panelArray_3011.add(labelArray_3011[k]);

}

panelArray_3011.revalidate();

panelArray_3011.repaint();

}

private void performStep_3011() {

    if (i_3011 < array_3011.length && sorting_3011) {

        int key_3011 = array_3011[i_3011];

        j_3011 = i_3011 - 1;

```

```

StringBuilder stepLog_3011 = new StringBuilder();

stepLog_3011.append("Langkah ").append(stepCount_3011)
    .append(": Memasukkan ").append(key_3011).append("\n");

while (j_3011 >= 0 && array_3011[j_3011] > key_3011) {
    array_3011[j_3011 + 1] = array_3011[j_3011];
    j_3011--;
}

array_3011[j_3011 + 1] = key_3011;

updateLabels_3011();

stepLog_3011.append("Hasil: ")
    .append(arrayToString(array_3011))
    .append("\n\n");

stepArea_3011.append(stepLog_3011.toString());

i_3011++;

stepCount_3011++;

```

```

        if (i_3011 == array_3011.length) {
            sorting_3011 = false;

            stepButton_3011.setEnabled(false);

            JOptionPane.showMessageDialog(this, "Sorting selesai!");
        }
    }
}

private void updateLabels_3011() {
    for (int k_3011 = 0; k_3011 < array_3011.length; k_3011++) {
        labelArray_3011[k_3011]
            .setText(String.valueOf(array_3011[k_3011]));
    }
}

private void reset_3011() {
    inputField_3011.setText("");
    panelArray_3011.removeAll();
    panelArray_3011.revalidate();
    panelArray_3011.repaint();
    stepArea_3011.setText("");
}

```

```
        stepButton_3011.setEnabled(false);

        sorting_3011 = false;

        i_3011 = 1;

        stepCount_3011 = 1;

    }

    private String arrayToString(int[] arr_3011) {

        StringBuilder sb_3011 = new StringBuilder();

        for (int k_3011 = 0; k_3011 < arr_3011.length; k_3011++) {

            sb_3011.append(arr_3011[k_3011]);

            if (k_3011 < arr_3011.length - 1)

                sb_3011.append(", ");

        }

        return sb_3011.toString();

    }

    public static void main(String[] args) {

        SwingUtilities.invokeLater(() -> {
```

```

        InsertionSortGUI_2511533011    gui    =    new
InsertionSortGUI_2511533011();

        gui.setVisible(true);

    });

}

}

```

Class InsertionSortGUI_2511533011 merupakan program Java GUI (Graphical User Interface) yang digunakan untuk menampilkan proses pengurutan data menggunakan algoritma **Insertion Sort** secara langkah demi langkah.

Program ini dibuat menggunakan library Swing sehingga pengguna dapat:

- memasukkan data array,
- melihat proses sorting secara visual,
- menjalankan sorting per langkah,
- serta mereset tampilan.

Class ini merupakan pengembangan dari program Insertion Sort biasa menjadi tampilan interaktif berbasis GUI.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma sorting digunakan untuk mengurutkan data agar lebih teratur dan mudah diolah. Pada praktikum ini telah berhasil dibuat program Insertion Sort, Selection Sort, dan Bubble Sort menggunakan bahasa pemrograman Java.

Setiap algoritma memiliki cara kerja yang berbeda. Insertion Sort mengurutkan data dengan menyisipkan elemen pada posisi yang tepat, Selection Sort memilih nilai terkecil pada setiap langkah, sedangkan Bubble Sort menukar elemen yang berdekatan hingga data terurut. Selain itu, pembuatan Insertion Sort berbasis GUI membantu memahami proses sorting secara lebih jelas karena langkah-langkah pengurutan dapat dilihat secara visual.

Melalui praktikum ini, praktikan dapat lebih memahami konsep dasar sorting, penggunaan perulangan dan array, serta penerapan Java Swing dalam membuat tampilan GUI sederhana.

3.2 Saran

Sebaiknya praktikan lebih sering mencoba berbagai jenis data dan jumlah data yang berbeda agar semakin memahami cara kerja algoritma sorting. Selain itu, praktikan juga dapat mencoba membandingkan kecepatan masing-masing algoritma untuk mengetahui metode mana yang lebih efisien.

Pada pengembangan berikutnya, program GUI dapat dibuat lebih menarik dengan tambahan animasi, warna, atau fitur pengurutan lainnya seperti Quick Sort dan Merge Sort agar pembelajaran menjadi lebih interaktif dan mudah dipahami.

DAFTAR PUSTAKA

- [1] D. Bali, "Mengenal Sorting Berserta Contoh Source Code Pada Struktur Data," Medium, 14 Desember 2019. [Online]. Available: <https://daismabali.medium.com/mengenal-sorting-berserta-contoh-source-code-pada-struktur-data-9f57f6b85ac8>. [Accessed 20 Mei 2026].

TUGAS PRAKTIKUM PEKAN 7

3.3 Mahasiswa_2511533011

```
package pekan7_2511533011;

public class Mahasiswa_2511533011 {
    private String nama_3011;
    private String nim_3011;
    private String prodi_3011;

    // Constructor
    public Mahasiswa_2511533011(String nama_3011, String nim_3011, String
prodi_3011) {
        this.nama_3011 = nama_3011;
        this.nim_3011 = nim_3011;
        this.prodi_3011 = prodi_3011;
    }

    // Getter dan Setter
    public String getNama_3011() {
        return nama_3011;
    }

    public void setNama_3011(String nama_3011) {
        this.nama_3011 = nama_3011;
    }

    public String getNim_3011() {
        return nim_3011;
    }

    public void setNim_3011(String nim_3011) {
        this.nim_3011 = nim_3011;
    }

    public String getProdi_3011() {
        return prodi_3011;
    }

    public void setProdi_3011(String prodi_3011) {
        this.prodi_3011 = prodi_3011;
    }

    @Override
    public String toString() {
        return nama_3011 + " | " + nim_3011 + " | " + prodi_3011;
    }
}
```

3.4 SortingGUI_2511533011

```
package pekan7_2511533011;

import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.util.ArrayList;

import javax.swing.*;

public class SortingGUI_2511533011 extends JFrame {

    private static final long serialVersionUID = 1L;

    // INPUT FIELD
    private JTextField namaField_3011;
    private JTextField nimField_3011;
    private JTextField prodiField_3011;

    // BUTTON
    private JButton tambahButton_3011;
    private JButton hapusButton_3011;
    private JButton sortButton_3011;

    // COMBO BOX
    private JComboBox<String> comboSort_3011;

    // TEXT AREA
    private JTextArea areaData_3011;
    private JTextArea areaLangkah_3011;

    // DATA
    private ArrayList<Mahasiswa_2511533011> list_3011;

    public SortingGUI_2511533011() {

        setTitle("GUI Sorting Mahasiswa");
        setSize(900, 500);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        list_3011 = new ArrayList<>();

        // PANEL INPUT
        JPanel panelInput_3011 = new JPanel(new FlowLayout());

        namaField_3011 = new JTextField(10);
        nimField_3011 = new JTextField(10);
        prodiField_3011 = new JTextField(10);
```

```

        tambahButton_3011 = new JButton("Tambah");
        hapusButton_3011 = new JButton("Hapus (1 Data)");
        sortButton_3011 = new JButton("Mulai Sorting");

        comboSort_3011 = new JComboBox<>();
        comboSort_3011.addItem("Insertion Sort");
        comboSort_3011.addItem("Selection Sort");
        comboSort_3011.addItem("Bubble Sort");

        panelInput_3011.add(new JLabel("Nama"));
        panelInput_3011.add(namaField_3011);

        panelInput_3011.add(new JLabel("NIM"));
        panelInput_3011.add(nimField_3011);

        panelInput_3011.add(new JLabel("Prodi"));
        panelInput_3011.add(prodiField_3011);

        panelInput_3011.add(tambahButton_3011);
        panelInput_3011.add(hapusButton_3011);
        panelInput_3011.add(comboSort_3011);
        panelInput_3011.add(sortButton_3011);

        // AREA
        areaData_3011 = new JTextArea();
        areaData_3011.setEditable(false);

        areaLangkah_3011 = new JTextArea();
        areaLangkah_3011.setEditable(false);

        JSplitPane splitPane_3011 = new JSplitPane(
            JSplitPane.HORIZONTAL_SPLIT,
            new JScrollPane(areaData_3011),
            new JScrollPane(areaLangkah_3011)
        );

        splitPane_3011.setDividerLocation(450);

        add(panelInput_3011, BorderLayout.NORTH);
        add(splitPane_3011, BorderLayout.CENTER);

        // EVENT
        tambahButton_3011.addActionListener(e -> tambahData_3011());
        hapusButton_3011.addActionListener(e -> hapusData_3011());
        sortButton_3011.addActionListener(e -> mulaiSorting_3011());
    }

    // ===== TAMBAH DATA =====
    private void tambahData_3011() {

```

```

        String nama_3011 = namaField_3011.getText().trim();
        String nim_3011 = nimField_3011.getText().trim();
        String prodi_3011 = prodiField_3011.getText().trim();

        if (nama_3011.isEmpty() || nim_3011.isEmpty() ||
prodi_3011.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Input tidak boleh
kosong!");
            return;
        }

        list_3011.add(new Mahasiswa_2511533011(nama_3011, nim_3011,
prodi_3011));
        tampilData_3011();

        namaField_3011.setText("");
        nimField_3011.setText("");
        prodiField_3011.setText("");
    }

    // ===== TAMPIL DATA =====
    private void tampilData_3011() {

        areaData_3011.setText("");

        for (Mahasiswa_2511533011 mhs_3011 : list_3011) {
            areaData_3011.append(mhs_3011.toString() + "\n");
        }
    }

    // ===== HAPUS 1 DATA (BERDASARKAN NIM) =====
    private void hapusData_3011() {

        String nimHapus_3011 = JOptionPane.showInputDialog(
            this, "Masukkan NIM yang ingin dihapus");

        if (nimHapus_3011 == null || nimHapus_3011.trim().isEmpty()) {
            return;
        }

        boolean ditemukan_3011 = false;

        for (int i_3011 = 0; i_3011 < list_3011.size(); i_3011++) {

            if (list_3011.get(i_3011)
                .getNim_3011()
                .equalsIgnoreCase(nimHapus_3011)) {

                list_3011.remove(i_3011);
                ditemukan_3011 = true;
            }
        }
    }

```

```

        break;
    }
}

if (ditemukan_3011) {
    JOptionPane.showMessageDialog(this, "Data berhasil dihapus");
    tampilData_3011();
} else {
    JOptionPane.showMessageDialog(this, "NIM tidak ditemukan");
}
}

// ===== SORTING =====
private void mulaiSorting_3011() {
    if (list_3011.size() <= 1) {
        JOptionPane.showMessageDialog(this, "Data belum cukup untuk
sorting!");
        return;
    }

    areaLangkah_3011.setText("");

    String pilihan_3011 = comboSort_3011.getSelectedItem().toString();

    if (pilihan_3011.equals("Insertion Sort")) {
        insertionSort_3011();
    } else if (pilihan_3011.equals("Selection Sort")) {
        selectionSort_3011();
    } else {
        bubbleSort_3011();
    }
}

// ===== INSERTION SORT =====
private void insertionSort_3011() {
    areaLangkah_3011.append("=== INSERTION SORT ===\n\n");

    new Thread(() -> {
        for (int i_3011 = 1; i_3011 < list_3011.size(); i_3011++) {
            Mahasiswa_2511533011 key_3011 = list_3011.get(i_3011);
            int j_3011 = i_3011 - 1;

            while (j_3011 >= 0 &&
                list_3011.get(j_3011).getNama_3011()
                    .compareToIgnoreCase(key_3011.getNama_3011()) > 0) {

```

```

        list_3011.set(j_3011 + 1, list_3011.get(j_3011));
        j_3011--;
    }

    list_3011.set(j_3011 + 1, key_3011);

    final int langkah_3011 = i_3011 + 1;

    SwingUtilities.invokeLater(() -> {
        tampilData_3011();
        areaLangkah_3011.append("Langkah " + langkah_3011 + " : " + tampilNama_3011() + "\n");
    });

    delay_3011();
}

}).start();
}

// ===== SELECTION SORT =====
private void selectionSort_3011() {

    areaLangkah_3011.append("=== SELECTION SORT ===\n\n");

    new Thread(() -> {

        for (int i_3011 = 0; i_3011 < list_3011.size() - 1; i_3011++) {

            int minIndex_3011 = i_3011;

            for (int j_3011 = i_3011 + 1; j_3011 < list_3011.size(); j_3011++) {

                if (list_3011.get(j_3011).getNama_3011().compareToIgnoreCase(list_3011.get(minIndex_3011).getNama_3011()) < 0) {
                    minIndex_3011 = j_3011;
                }

                Mahasiswa_2511533011 temp_3011 = list_3011.get(i_3011);
                list_3011.set(i_3011, list_3011.get(minIndex_3011));
                list_3011.set(minIndex_3011, temp_3011);

                final int langkah_3011 = i_3011 + 1;

                SwingUtilities.invokeLater(() -> {
                    tampilData_3011();

```

```

        areaLangkah_3011.append("PASS " + langkah_3011 + " : "
+ tampilNama_3011() + "\n");
    });

    delay_3011();
}

}).start();
}

// ===== BUBBLE SORT =====
private void bubbleSort_3011() {

    areaLangkah_3011.append("=== BUBBLE SORT ===\n\n");

    new Thread(() -> {

        for (int i_3011 = 0; i_3011 < list_3011.size() - 1; i_3011++) {

            for (int j_3011 = 0; j_3011 < list_3011.size() - i_3011 -
1; j_3011++) {

                if (list_3011.get(j_3011).getNama_3011()
                    .compareToIgnoreCase(list_3011.get(j_3011 +
1).getNama_3011()) > 0) {

                    Mahasiswa_2511533011 temp_3011 =
list_3011.get(j_3011);
                    list_3011.set(j_3011, list_3011.get(j_3011 + 1));
                    list_3011.set(j_3011 + 1, temp_3011);
                }
            }

            final int langkah_3011 = i_3011 + 1;

            SwingUtilities.invokeLater(() -> {
                tampilData_3011();
                areaLangkah_3011.append("PASS " + langkah_3011 + " : "
+ tampilNama_3011() + "\n");
            });

            delay_3011();
        }

    }).start();
}

// ===== DELAY =====
private void delay_3011() {
    try {

```

```

        Thread.sleep(1000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}

// ===== TAMPIL NAMA =====
private String tampilNama_3011() {

    String hasil_3011 = "[";

    for (int i_3011 = 0; i_3011 < list_3011.size(); i_3011++) {

        hasil_3011 += list_3011.get(i_3011).getNama_3011();

        if (i_3011 != list_3011.size() - 1) {
            hasil_3011 += ", ";
        }

    }

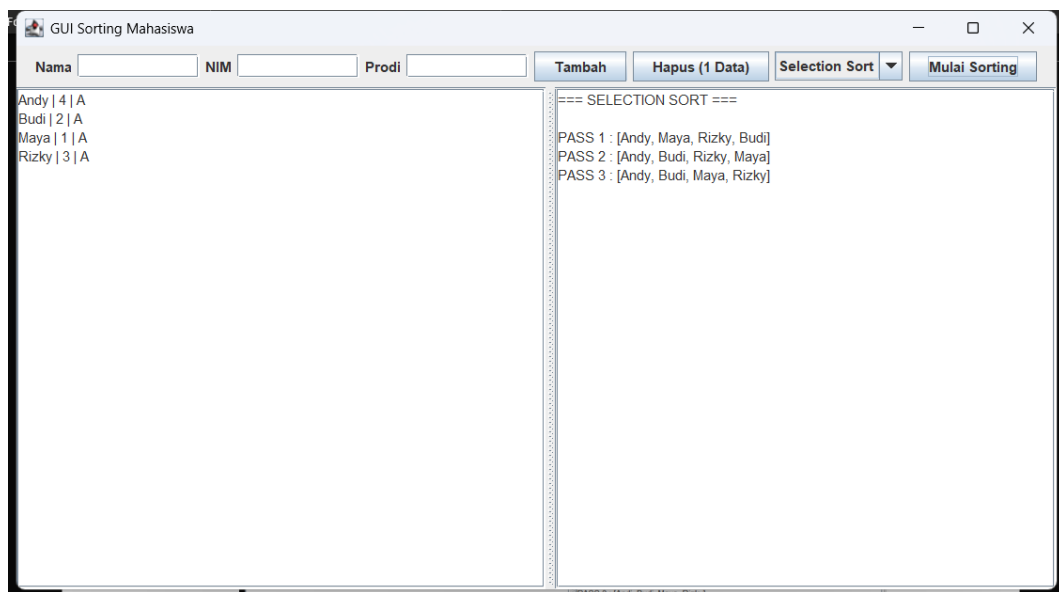
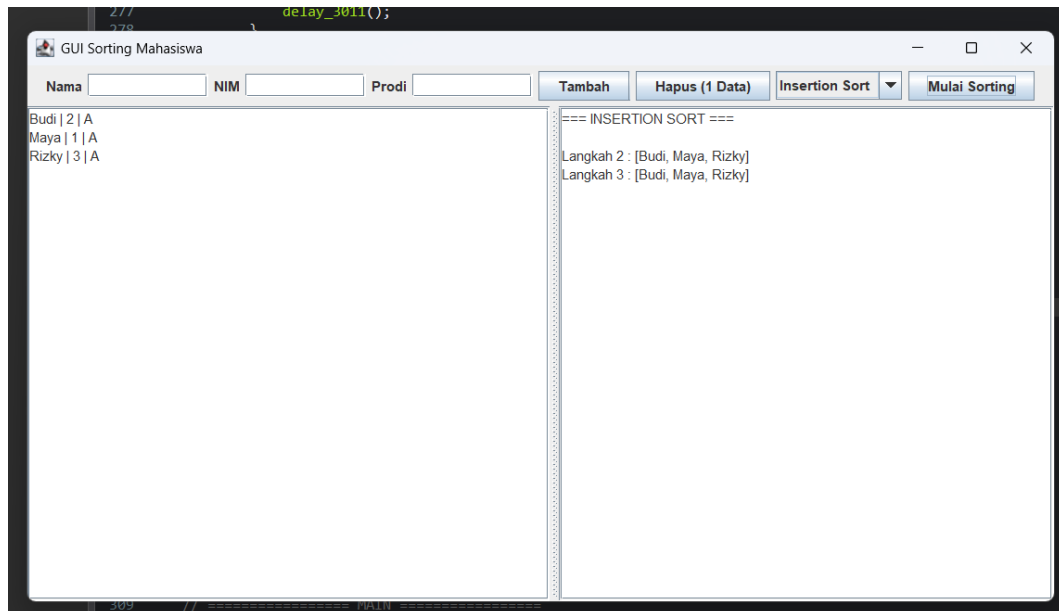
    return hasil_3011 + "]";
}

// ===== MAIN =====
public static void main(String[] args) {

    SwingUtilities.invokeLater(() -> {
        new SortingGUI_2511533011().setVisible(true);
    });
}
}

```

Output :



GUI Sorting Mahasiswa

Nama NIM Prodi

Andy | 4 | A
Budi | 2 | A
Maya | 1 | A
Rizky | 3 | A

=== BUBBLE SORT ===
PASS 1 : [Andy, Maya, Budi, Rizky]
PASS 2 : [Andy, Budi, Maya, Rizky]
PASS 3 : [Andy, Budi, Maya, Rizky]